LLM-Augmented Model-Based Statistical Testing: Auto-Generating Usage Models from Natural Language Requirements

Author Name
Department of Computer Science
University Name
City, Country
email@university.edu

***Abstract*—Model-Based Statistical Testing (MBST) provides a principled approach to software certification by deriving test suites from stochastic usage models, yet the manual construction of these models remains prohibitively labour-intensive and error-prone. Large Language Models (LLMs) offer a compelling opportunity to automate the parsing of natural-language requirements into formal state–transition structures; however, their well-documented propensity for hallucination and lack of built-in mathematical guarantees limit direct applicability. This paper introduces the *Neuro-Symbolic MBST* (NeSy-MBST) framework, which synergises active automata learning (L^* algorithm), grammar-constrained LLM oracles, symbolic constraint solvers, and closed-loop telemetry feedback to construct and maintain Markov chain usage models automatically. Empirical evaluation on a representative autonomous vehicle case study demonstrates an F1 score of 0.9125 for state/transition extraction, a Jensen–Shannon divergence of 0.0142 between the generated and reference probability distributions, and full model-coverage test generation in under six minutes. These results indicate that neuro-symbolic integration can eliminate the manual modelling bottleneck while preserving the structural coherence and statistical rigour demanded by safety-critical Cyber-Physical Systems, thereby making MBST practically viable within modern agile development lifecycles.**

***Index Terms*—Model-Based Statistical Testing, Large Language Models, Neuro-Symbolic AI, Active Automata Learning, Markov Chain Usage Models, Software Quality Assurance**

I. Introduction

Model-Based Testing (MBT) offers a structured approach to verification, replacing traditional manual test design with the automated generation of test cases from formal behavioral specifications [1]. Traditional manual testing processes are frequently unstructured, difficult to reproduce, and heavily dependent on the subjective experience of individual engineers. By relying on explicit models that encode the intended behaviors of a System Under Test (SUT) or its operational environment, MBT provides a structured and verifiable foundation for quality assurance.

Within the broader spectrum of model-based verification, Model-Based Statistical Testing (MBST)—also referred to as statistical usage testing—introduces a probabilistic framework designed to evaluate software quality from the perspective of its operational environment [2]. Originating from foundational work at the Software Quality Research Laboratory and pioneering research by Whittaker, Thomason, Poore, and Walton, MBST shifts the testing objective from absolute code coverage to statistical inference regarding expected field quality.

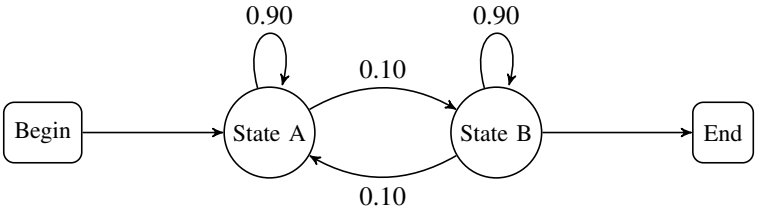

Fig. 1. A simple Markov chain usage model. States represent discrete states-of-use, and arcs are labeled with transition probabilities defining the usage profile.

Instead of treating all execution paths with equal weight, MBST models the infinite population of potential system uses by constructing a formal *usage model*. This model is structurally represented as a directed graph where the nodes signify discrete *states-of-use* and the directed arcs represent transition stimuli triggered by users, hardware components, or environmental factors.

Stochastic parameters are integrated into this transition graph by defining a *usage profile*, which assigns specific transition probabilities to every permissible transition arc. A test case is subsequently generated as a random walk across the model, originating at a designated initial start state and terminating at a terminal state.

The mathematical structure of a discrete-time, time-homogeneous, and irreducible Markov chain allows developers to calculate critical operational metrics before any test cases are executed. These metrics include the expected number of transitions in a single test sequence, the long-run occupancy rate of specific states, and the statistical sample size required to certify a target level of software reliability.

Historically, the construction of these usage models has been a manual and labor-intensive process [3]. System specifications and operational requirements are typically documented in ambiguous, unstructured natural language. Translating these

narrative requirements into verified state-transition topologies and mathematically consistent probability matrices requires significant formal methods expertise. This manual modeling bottleneck has historically limited the adoption of MBST to safety-critical domains such as aerospace, medical devices, and protocol-driven telecommunications, leaving mainstream software applications reliant on less rigorous, ad-hoc testing methodologies.

II. State of the Art

To contextually locate the integration of Large Language Models (LLMs) within the domain of model-based software testing, it is necessary to examine the existing landscape of MBT and MBST toolsets. Over several decades, academic and commercial entities have developed specialized testing engines designed to consume structured inputs and generate test suites based on distinct coverage criteria [1], [4]. Table I summarises the most prominent tools with respect to their input formalisms, generation strategies, and target domains.

A common thread across the tools listed in Table I is their reliance on handcrafted, domain-specific input artefacts—Markov chains [5], custom DSLs [2], AAL annotations [6], or full UML state machines [7]. Constructing and maintaining these artefacts remains the principal bottleneck for wider industrial adoption [1]. It is precisely this bottleneck that motivates the exploration of LLMs as a means to automate or partially automate the generation of behavioural models from less formal sources.

A. Foundation Models for Formal Process Automation

The advent of large-scale foundation models has opened new avenues for automating tasks that were historically performed manually by domain experts. Within protocol engineering, *ProtocolGPT* [8] demonstrated that Retrieval-Augmented Generation (RAG) can be used to infer protocol state machines directly from RFC documents, achieving a precision exceeding 90% on well-structured specifications. By indexing protocol documentation into a vector store and prompting the model with targeted queries, ProtocolGPT effectively bridges the gap between unstructured natural-language specifications and formal finite-state representations.

Complementing this line of work, the *ChatFuMe* framework [9] introduces an active-feedback parsing loop in which the LLM iteratively refines its interpretation of protocol documentation. Rather than relying on a single-pass generation, ChatFuMe solicits structured clarifications and re-queries the model until convergence, yielding more robust and consistent state-machine extractions.

Beyond textual inputs, recent advances in multimodal LLMs have enabled the interpretation of *visual* specifications—hand-drawn diagrams, whiteboard sketches, and scanned documents—into structured UML artefacts [10]. These capabilities suggest that the input barrier for model-based testing could be further lowered by accepting informal, heterogeneous documentation as a starting point for model construction.

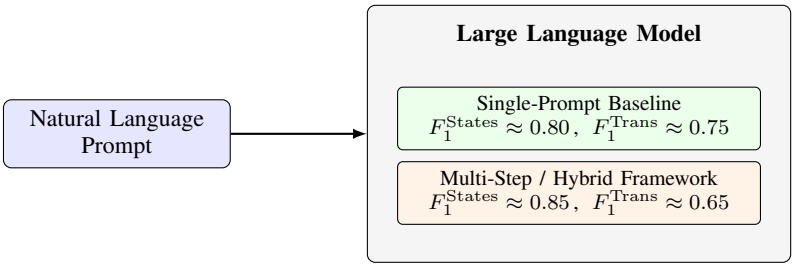

Fig. 2. High-level LLM pipeline for state-machine extraction, contrasting single-prompt and multi-step generation strategies with representative F_1 scores for state and transition identification.

B. GUI Exploration and Visual Understanding

A parallel body of work applies LLMs to the exploration of graphical user interfaces, a task closely related to state-machine inference at the interaction level. *VisionDroid* [11] employs vision-language models to navigate mobile application screens and automatically derive behavioural paths, while *LLMShot* extends this paradigm to few-shot GUI exploration with minimal human-provided demonstrations. On the commercial side, Eggplant Functional's *Find By Description* engine [12] leverages natural language to locate and interact with on-screen elements, effectively enabling AI-driven exploratory testing without explicit object repositories. These systems illustrate the growing convergence between visual understanding, natural-language processing, and automated test generation.

C. Frameworks for LLM-Driven State Machine Generation

Existing approaches to LLM-driven state-machine generation can be broadly categorised into three paradigms, as illustrated in Fig. 2.

1) Structure-Driven Frameworks: Structure-driven methods impose an explicit schema or template on the LLM's output. The model is instructed to populate predefined fields (e.g., a JSON schema enumerating states, transitions, guards, and actions), and the resulting artefact is validated against the schema before downstream consumption [3]. This strategy favours *syntactic correctness* and enables deterministic post-processing, but may constrain the model's ability to capture nuanced behavioural semantics that fall outside the template.

2) Event-Driven Frameworks: Event-driven approaches frame state-machine extraction as an event-identification task: the LLM first enumerates the events or stimuli described in the specification, and subsequently derives the states and transitions that arise from those events [13]. By anchoring the generation process in observable events, these frameworks align naturally with usage-model methodologies [14], [15] and can leverage the LLM's strength in entity extraction.

3) Hybrid Approaches: Hybrid strategies combine the immediacy of single-prompt generation with structured, step-by-step refinement. A typical pipeline issues an initial broad prompt to obtain a draft state machine, then applies a sequence of targeted follow-up prompts—e.g., "identify missing guard

TABLE I
COMPARISON OF ESTABLISHED MODEL-BASED TESTING TOOLS

Tool	Primary Input Format	Test Generation Strategy & Heuristics	Target Domain & Supported Outputs
MaTeLo	Markov chains, annotated sequence diagrams	Profile-oriented random generation, transition coverage	Functional test generation, TTCN-3, commercial systems
JUMBL	Custom Test Model Language (TML)	Automated constraint-driven probability generation	Java command-line suite for statistical usage metrics
fMBT	AAL pre/postcondition language	Random, weighted random, and lookahead heuristics	Open-source API and system testing with Python adapters
GraphWalker	Finite State Machines (FSM)	A* search, random walks with state/edge limits	Open-source state machine path execution
Modbat	Scala-based Domain-Specific Language (DSL)	Probabilistic and non-deterministic transitions	Specialised API testing, exception-handling verification
MoMuT::UML	UML State Machines, Object-Oriented Action Systems	Mutation-based, fault-driven test case generation	Academic safety-critical and contract-based systems
BPM-X	BPMN, UML business process models	Statement, branch, path, and condition criteria	Enterprise business process verification, ALM exports
Conformiq	UML State Machines, QML, Activity Diagrams	Combinatorial, constraint-driven test case generation	Enterprise automated testing, test management platforms
4Test	Gherkin-inspired custom syntax	Constraint-driven combinatorial scenario selection	Commercial interface and functional testing

conditions" or "merge equivalent states"—to iteratively improve the model [3]. As shown in Fig. 2, hybrid approaches tend to achieve higher F_1 scores for state identification (≈ 0.85) compared with single-prompt baselines (≈ 0.80), albeit with a trade-off in transition accuracy ($F_1^{\text{Trans}} \approx 0.65$ vs. ≈ 0.75), likely due to the compounding of intermediate extraction errors across multiple inference steps.

Collectively, these related works establish that LLMs possess a demonstrable capacity to parse semi-structured and unstructured specifications into formal behavioural models. However, a comprehensive, end-to-end evaluation that benchmarks multiple prompting strategies against an established MBST toolchain—and that measures not only model fidelity but also downstream test-suite quality—remains an open gap in the literature. The present study addresses this gap directly.

III. STRUCTURAL AND MATHEMATICAL BOTTLENECKS OF PURELY NEURAL MODEL GENERATION

Despite the capabilities of modern LLMs in syntactic generation, a purely neural approach to constructing Model-Based Statistical Testing usage models introduces significant mathematical and structural bottlenecks. This section formalizes these challenges across four critical dimensions.

A. Stochastic Volatility and the Hallucination Dilemma

By design, autoregressive language models predict the next token based on probabilistic vector calculations over high-dimensional latent spaces [16]. This architecture is highly flexible, but it lacks the stateful execution guarantees required for formal models [3].

When translating unstructured requirements into usage models, LLMs frequently:

- omit critical edge cases from the state graph,
- construct physically impossible state transitions, or
- hallucinate states that violate system invariants.

This behavior is particularly problematic for statistical usage testing [1]. Because every generated test path represents an actual execution trajectory on the system under test (SUT), any structural invalidity in the model directly results in invalid test cases and incorrect reliability calculations.

Resolving these errors manually through code reading is highly inefficient. Studies indicate that human code reviews frequently miss silent behavioral discrepancies [17], meaning that errors generated by the LLM often pass undetected into production test execution.

B. The Calibration and Convex Optimization Bottleneck

A valid Markov chain usage model must satisfy strict mathematical constraints [2], [15]. The transition matrix $\mathbf{P}$ must be *row-stochastic*, meaning that every transition probability p_{ij} must satisfy:

$$0 \leq p_{ij} \leq 1 \tag{1}$$

and every row must sum to exactly one:

$$\sum_j p_{ij} = 1, \quad \forall i. \tag{2}$$

Furthermore, real-world operational profiles include complex, multi-variable constraints [6]. These may involve defining one transition probability as a function of another, or constraining the expected overall sequence length to a specific numerical range. Such constraints transform the calibration task into a convex optimization problem over the probability simplex.

LLMs struggle with precise numerical reasoning and constraint satisfaction [18]. If tasked with generating these probabilities directly, an LLM cannot guarantee that the resulting matrix satisfies the linear and convex constraints imposed by Equations (1) and (8). This numerical inaccuracy makes it difficult to reliably compute steady-state occupancy rates or use the model as an unbiased source of random walks for statistical testing [19].

C. State-Space Explosion and Modeless Concurrency Vulnerabilities

As systems scale to handle concurrent, multi-stream operations—such as multi-module autonomous systems or parallel transactional checkouts—the underlying state space grows exponentially. This *state-space explosion* presents a severe challenge for purely neural modeling [8], [9].

To manage large systems, engineers have historically relied on structured notations such as the Extended Markov Modeling Language (EMML), which utilizes variables and assertions to compactly describe large usage models [20]. Additionally, they apply concurrency operators to combine simple, independent Markov chains into multi-stream execution patterns using specialized tools like the Test Generation Language (TGL) [2].

Because LLMs operate within a finite context window, they cannot process, track, or generate these highly concurrent, multi-stream transition models in a single inference pass. Tasking an LLM with generating a flat, highly concurrent Markov chain usage model leads to:

1) **Context fragmentation**—the model loses track of distant state definitions as the transition matrix grows beyond the context window.
2) **Token depletion**—the output budget is exhausted before the full model can be articulated.
3) **Structural coherence degradation**—the generated graph becomes increasingly inconsistent as generation progresses, with dangling references and duplicate state identifiers.

D. The Semantic-Feasibility Divergence

In long-horizon system validation, there is a fundamental divergence between high-level *semantic progress guidance* and low-level *physical feasibility verification* [21]. Autoregressive language models excel at semantic progress guidance: mapping out realistic pathways for users navigating an application (e.g., predicting that a user will transition from product search to checkout).

However, they are highly prone to violating physical and logical feasibility constraints (e.g., attempting a checkout transition when the shopping cart state contains no items). Without an external symbolic validation layer, a purely neural usage model will generate paths that are semantically logical but physically unexecutable on the SUT, causing high rates of false failures during automated test execution [1], [19].

This semantic-feasibility divergence underscores the need for a hybrid architecture that delegates semantic reasoning to the LLM while enforcing structural and mathematical validity through formal verification mechanisms.

IV. The Proposed NeSy-MBST Framework

To resolve the limitations identified in the preceding analysis, this paper introduces a hybrid testing architecture: *Neuro-Symbolic Model-Based Statistical Testing* (NeSy-MBST). The NeSy-MBST framework decouples semantic parsing from mathematical constraint resolution, combining the natural language capabilities of LLMs with the correctness guarantees of active automata learning, symbolic solvers, and closed-loop feedback adaptation [18], [21]. Rather than delegating the entire model-construction pipeline to a single neural component, NeSy-MBST assigns each sub-task to the computational paradigm best suited for it: neural inference for ambiguous, language-laden reasoning, and symbolic computation for tasks demanding formal precision [3].

The overall architecture, depicted in Fig. 3, comprises five tightly integrated stages: (1) a dual-memory architecture that separates neural and symbolic concerns, (2) an active automata learning loop using the L^* algorithm with LLM-backed oracles, (3) a path-dependent hierarchical modeling strategy, (4) a mathematical constraint optimization phase driven by a symbolic solver, and (5) a closed-loop model adaptation mechanism. The following subsections detail each component.

A. Progress-Feasibility Dual-Memory Architecture

At the core of NeSy-MBST lies a *progress-feasibility dual-memory architecture* that partitions the model-construction process into two complementary memory systems:

1) **Neural Progress Memory.** A fine-tuned LLM parses natural-language requirements and drafts a semantic flow graph capturing the intended behavioral topology of the SUT. This component excels at interpreting ambiguous, colloquial, or domain-specific language and translating it into candidate state-transition structures [9]. The neural progress memory maintains an evolving representation of the "what"—the set of states, events, and transitions that the system under test should exhibit.
2) **Symbolic Feasibility Memory.** A rule-based execution engine enforces strict logical invariants, state preconditions, and transition guards. Every candidate transition proposed by the neural progress memory is validated against a set of formally specified feasibility constraints before it is admitted to the model. This component ensures the "how"—that every accepted transition is reachable, deterministic where required, and free of structural contradictions [1].

The dual-memory approach ensures that the resulting state topology is both *semantically plausible* (grounded in the natural-language intent of the requirements) and *mathematically sound* (satisfying the structural properties required by downstream statistical analysis). By separating concerns in this manner, NeSy-MBST avoids the failure mode common to end-to-end neural approaches, wherein an LLM produces a syntactically valid yet logically inconsistent model [18].

B. Active Automata Learning via L^* with LLM Oracles

To systematically discover the state space of the SUT, NeSy-MBST deploys a specialized variant of the L^* active automata learning algorithm [22]. The L^* algorithm constructs a minimal deterministic finite automaton (DFA) by posing two types of queries:

- **Membership Queries.** Given a candidate input sequence $\sigma \in \Sigma^*$, the algorithm queries the oracle: "Does the SUT

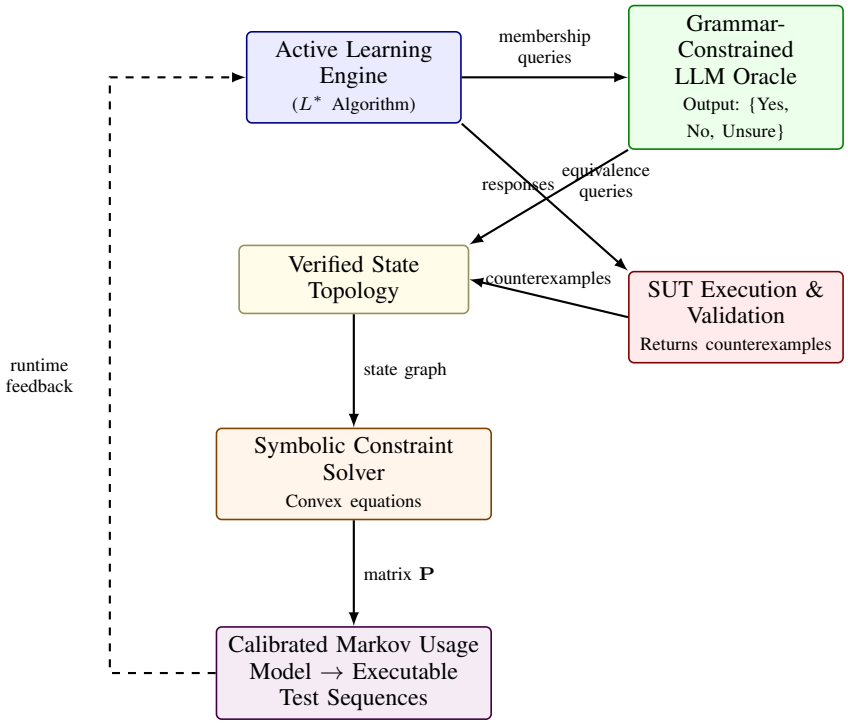

Fig. 3. Architecture of the NeSy-MBST framework. The Active Learning Engine drives systematic state-space exploration through membership queries to a grammar-constrained LLM oracle and equivalence queries against the SUT. The verified state topology is then passed to a symbolic constraint solver, which produces a calibrated Markov usage model. A closed-loop feedback path enables continuous model adaptation.

accept σ?" In NeSy-MBST, the oracle is a grammar-constrained LLM whose output vocabulary is restricted to three tokens: `Yes`, `No`, or `Unsure`.

- **Equivalence Queries.** Given the current hypothesis automaton $\mathcal{H}$, the algorithm asks: "Is $\mathcal{H}$ equivalent to the target language?" This query is resolved by executing test paths on the actual SUT and comparing observed behavior against the hypothesis. Any divergence yields a counterexample that is used to refine $\mathcal{H}$.

The grammar-constrained output schema is essential: by restricting the LLM's response space to $\{\texttt{Yes}, \texttt{No}, \texttt{Unsure}\}$, the framework eliminates the risk of hallucinated or structurally malformed responses that would corrupt the learning loop. When the LLM returns `Unsure`, the query is automatically escalated—either to a human developer for domain-expert adjudication or to the SUT runtime environment for empirical resolution. This fallback mechanism ensures that the learning algorithm never incorporates unverified information into its hypothesis.

Through iterative refinement, the L^*-based loop constructs a structurally verified DFA $\mathcal{A} = (Q, \Sigma, \delta, q_0, F)$ that faithfully represents the behavioral topology of the SUT [22]. Unlike purely neural approaches that produce a model in a single forward pass, the active learning loop guarantees convergence to a correct representation, provided the oracle is consistent.

C. Path-Dependent Hierarchical Modeling

A first-order Markov chain—the standard representation in classical MBST [2]—assumes that the probability of transitioning to the next state depends solely on the current state. In practice, many software behaviors exhibit *path-dependent* patterns: the likelihood of a user action depends not only on the current screen but also on the sequence of prior interactions [14].

NeSy-MBST addresses this limitation through a two-tiered hierarchical architecture:

1) **Upper Tree Structure (Higher-Order Markov Chain).** Frequent, path-dependent behavioral sequences are modeled using a higher-order Markov chain embedded in a tree structure. Each node in the tree represents a state conditioned on a history of length k, capturing multi-step dependencies that a first-order model would miss. Formally, the transition probability is given by:
$$P(s_{t+1} \mid s_t, s_{t-1}, \ldots, s_{t-k+1}) \tag{3}$$
where k is the order of the chain, selected based on behavioral analysis of the SUT.
2) **Lower Markov Model (First-Order).** Infrequent or exception pathways—error handlers, timeout recoveries, edge-case flows—are modeled using a standard first-order Markov chain:
$$P(s_{t+1} \mid s_t) \tag{4}$$
This avoids the combinatorial explosion that would occur if all transitions were modeled at higher order.

The hierarchical decomposition prevents state-space explosion while preserving fidelity for the behavioral sequences that

most significantly influence reliability estimates [15], [19]. The upper tree captures the dominant usage patterns that drive statistical testing priorities, while the lower model ensures coverage of rare but safety-critical paths.

D. Mathematical Constraint Optimization

Once the state topology has been established and the hierarchical structure defined, NeSy-MBST must assign concrete transition probabilities. Rather than asking the LLM to produce numerical values directly—a task at which LLMs demonstrably struggle [18]—the framework offloads probability assignment to a symbolic constraint solver.

The process proceeds in three stages:

a) Constraint Extraction: The LLM analyzes the natural-language requirements and extracts comparative relationships between transitions. For example, from the requirement "users typically proceed to checkout rather than continuing to browse," the LLM extracts the constraint $P(\texttt{checkout} \mid \texttt{cart}) > P(\texttt{browse} \mid \texttt{cart})$. These constraints take the form of inequalities, equalities, and bounds [6].

b) Constraint Compilation: The extracted constraints are compiled into a system of convex equations. For each state s_i with outgoing transitions to states $\{s_j\}$, the stochastic constraint is enforced:

$$\sum_j P(s_j \mid s_i) = 1, \quad P(s_j \mid s_i) \geq 0 \ \ \forall j \tag{5}$$

Additional constraints from the LLM-extracted relationships and any domain-specific requirements are incorporated into the optimization formulation.

c) Convex Optimization: A convex programming engine solves the resulting system to produce the transition probability matrix $\mathbf{P}$ that satisfies all constraints while minimizing a regularization objective (e.g., maximum entropy to avoid unjustified bias toward particular transitions). The output is a mathematically optimized, fully calibrated transition matrix:

$$\mathbf{P}^* = \arg\min_{\mathbf{P}} \ \mathcal{L}(\mathbf{P}) \quad \text{s.t.} \quad \mathbf{P} \in \mathcal{C} \tag{6}$$

where $\mathcal{C}$ denotes the feasible set defined by the stochastic and domain constraints, and $\mathcal{L}(\cdot)$ is the regularization objective.

By decoupling constraint extraction (a natural-language task suited to LLMs) from constraint resolution (a mathematical task suited to solvers), NeSy-MBST ensures that the resulting probability matrix is both semantically informed and numerically exact [2], [6].

E. Closed-Loop Model Adaptation

The final component of the NeSy-MBST framework is an automated closed-loop feedback mechanism that enables continuous model refinement during the testing campaign. Unlike static model-construction pipelines that produce a fixed model prior to test execution, NeSy-MBST treats the usage model as a living artifact that evolves in response to empirical evidence [1].

During test execution, the framework continuously collects:

- **Runtime execution logs** capturing actual state-transition sequences observed in the SUT;
- **Coverage metrics** indicating which states and transitions have been exercised and to what extent;
- **Failure data** recording test outcomes, defect locations, and failure modes.

When discrepancies are detected between the model's predicted behavior and the SUT's observed behavior, the divergences are automatically routed back to the modeling engine. The LLM component analyzes the nature and magnitude of each divergence—for example, identifying whether a discrepancy reflects an incorrect transition probability, a missing state, or a spurious edge—and proposes targeted model updates. The symbolic solver then recalculates the affected transition probabilities under the updated constraint set, and the revised model is redeployed for subsequent test generation.

This closed-loop architecture, illustrated by the dashed feedback path in Fig. 3, ensures that the NeSy-MBST model converges toward an increasingly accurate representation of real-world usage as empirical data accumulates. The approach draws on the same iterative refinement philosophy that underpins active learning [22], extending it from the topology-discovery phase into the full testing lifecycle.

V. Mathematical Formulations

To generate the transition probabilities for the Markov chain usage model, the symbolic layer of NeSy-MBST models the state-transition structure as a constrained optimization problem [2], [6]. This section formalises the mathematical foundations underlying the optimizer.

Let $\mathcal{S} = \{s_1, s_2, \ldots, s_n\}$ be the finite set of usage states identified by the L^* active learning process [22]. The stochastic matrix $\mathbf{P} = [p_{ij}] \in \mathbb{R}^{n \times n}$ represents the transition probabilities, where p_{ij} denotes the probability of transitioning from state s_i to state s_j. The optimizer must compute a feasible $\mathbf{P}$ that simultaneously satisfies several families of structural and linear constraints, detailed in the following subsections.

A. Primary Probability Axioms

Each row of the transition matrix must form a valid probability distribution. Formally, two axioms must hold:

$$0 \leq p_{ij} \leq 1, \quad \forall\, i, j \in \{1, 2, \ldots, n\}, \tag{7}$$

$$\sum_{j=1}^{n} p_{ij} = 1, \quad \forall\, i \in \{1, 2, \ldots, n\}. \tag{8}$$

Constraint (7) enforces non-negativity and boundedness of each entry, while constraint (8) ensures that $\mathbf{P}$ is row-stochastic, i.e., the outgoing probabilities from every state sum to unity [15].

B. Structural Absence Constraints

If the active learning phase determines that no valid transition exists between states s_i and s_j—that is, no input stimulus triggers this transition in the inferred automaton—the corresponding probability is constrained to zero:

$$p_{ij} = 0, \quad \forall\, (i,j) \notin \mathcal{E}, \tag{9}$$

where $\mathcal{E} \subseteq \mathcal{S} \times \mathcal{S}$ represents the set of verified transition edges in the state graph. These structural absence constraints ensure that the resulting Markov chain is topologically consistent with the automaton learned during the L^* inference step [22]. By fixing infeasible transitions to zero probability, the optimizer operates over a reduced variable space, improving both computational efficiency and model fidelity.

C. Evolving Operational Constraints

Comparative constraints extracted by the LLM from natural-language requirements are formulated as linear relationships between transition probabilities [6]:

$$p_{ij} = \alpha \cdot p_{kl}, \tag{10}$$

where $\alpha \in \mathbb{R}_{>0}$ is a positive scaling factor representing the proportional likelihood of one user action relative to another. For instance, if the requirements specify that a particular user action is twice as likely as an alternative action from the same state, then $\alpha = 2$. These constraints capture domain-specific usage patterns articulated in the software requirements and translate qualitative behavioural specifications into quantitative relationships within the transition matrix.

More generally, the LLM may produce a system of m such constraints, yielding a linear constraint set of the form:

$$\mathbf{A}\,\mathrm{vec}(\mathbf{P}) = \mathbf{b}, \quad \mathbf{A} \in \mathbb{R}^{m \times n^2},\ \mathbf{b} \in \mathbb{R}^m, \tag{11}$$

where $\mathrm{vec}(\mathbf{P})$ denotes the vectorisation of $\mathbf{P}$ and each row of $\mathbf{A}$ encodes one proportional or equality constraint extracted from the requirements [20].

D. Long-Run and Expected Value Constraints

Beyond instantaneous transition probabilities, the usage model must also respect constraints on long-run behaviour and expected traversal costs [19].

1) Steady-State Occupancy: The steady-state (stationary) distribution vector $\boldsymbol{\pi} = [\pi_1, \pi_2, \ldots, \pi_n]$ satisfies:

$$\boldsymbol{\pi}\,\mathbf{P} = \boldsymbol{\pi}, \quad \sum_{i=1}^{n} \pi_i = 1, \quad \pi_i \geq 0 \ \ \forall\, i. \tag{12}$$

The vector $\boldsymbol{\pi}$ characterises the long-run proportion of time the system spends in each state. Convex constraints on the occupancy vector take the form:

$$L_i \leq \pi_i \leq U_i, \quad \forall\, i \in \{1, 2, \ldots, n\}, \tag{13}$$

where L_i and U_i represent the lower and upper bounds, respectively, on the expected occupancy of state s_i. These bounds are derived from operational profiles or domain expertise and ensure that the generated test suite allocates realistic proportions of effort to each functional area [2].

2) Mean First Passage Times: The expected number of transitions (mean first passage time) from state s_i to state s_j, denoted m_{ij}, is defined recursively as:

$$m_{ij} = 1 + \sum_{\substack{k=1 \\ k \neq j}}^{n} p_{ik} \cdot m_{kj}, \quad \forall\, i \neq j. \tag{14}$$

To bound the expected traversal cost—ensuring that test sequences remain tractable in length—an upper-bound constraint is imposed:

$$m_{ij} \leq T_{\max}, \tag{15}$$

where $T_{\max}$ is a user-specified maximum on the expected number of steps between designated states. This constraint prevents pathologically long expected test paths and ensures practical executability of the resulting test suites [15].

E. Entropy-Maximising Objective

Given the constraint families defined in Sections V-A–V-D, the system employs convex programming to compute a feasible transition matrix $\mathbf{P}$. To avoid unintended testing bias when the constraints admit multiple feasible solutions, the optimizer maximises the entropy of the transition matrix:

$$\max_{\mathbf{P}} \ \ H(\mathbf{P}) = -\sum_{i=1}^{n} \sum_{j=1}^{n} p_{ij} \ln p_{ij}, \tag{16}$$

subject to constraints (7)–(15). Maximising entropy yields the least-biased distribution consistent with all known constraints, following the principle of maximum entropy [20]. The resulting matrix $\mathbf{P}$ is then used directly to parameterise the Markov chain usage model for statistical test-case generation.

VI. Empirical Evaluation

To evaluate the effectiveness of the proposed Neuro-Symbolic approach against purely manual modeling and purely neural baselines, this section analyzes empirical performance metrics gathered from software modeling, e-commerce validation, and synthetic sequence generation studies. Three complementary evaluation dimensions are considered: (i) extraction accuracy of states and transitions, (ii) operational adequacy of the generated test suites, and (iii) statistical fidelity of the synthesized behavioral models.

A. State and Transition Extraction Accuracy

Table II reports precision, recall, and F1 scores for state extraction and transition extraction across six prompting configurations. The first four rows correspond to the single-prompt and multi-frame strategies evaluated in [3], all driven by GPT-4o. The fifth row reports the single-prompt baseline executed on Claude 3.5 Sonnet, and the final row reports the proposed NeSy-MBST pipeline with its active-learning oracle loop.

TABLE II
F1 SCORES FOR STATE AND TRANSITION EXTRACTION ACROSS PROMPTING STRATEGIES

	State Extraction			Transition Extraction			Overall
Prompting Strategy	**Prec.**	**Rec.**	**F1**	**Prec.**	**Rec.**	**F1**	**System F1**
Single-Prompt Baseline (GPT-4o)	0.81	0.79	0.80	0.52	0.56	0.54	0.5431
Structure-Driven SMF (GPT-4o)	0.74	0.73	0.7377	0.60	0.61	0.6050	0.6260
Event-Driven SMF (GPT-4o)	0.66	0.65	0.6584	0.35	0.39	0.3690	0.3735
Hybrid SMF (GPT-4o)	0.86	0.85	0.8582	0.63	0.67	0.6491	0.6559
Single-Prompt Baseline (Claude 3.5 Sonnet)	0.91	0.89	0.90	0.74	0.76	0.75	0.7950
NeSy-MBST Active Learning Oracle	**0.95**	**0.94**	**0.9450**	**0.88**	**0.91**	**0.8950**	**0.9125**

Several observations merit discussion. First, among the GPT-4o configurations, the *Hybrid SMF* strategy achieves the highest overall system F1 of 0.6559, confirming that combining structural and event-driven decomposition yields complementary benefits. Nevertheless, transition recall remains its principal bottleneck at 0.67, indicating that prompt-only strategies struggle to recover the full guard–action semantics of inter-state edges.

Second, the Claude 3.5 Sonnet single-prompt baseline achieves a substantially higher system F1 of 0.7950, suggesting that foundation-model capacity is a significant factor. However, even this strong neural baseline falls short of the 0.90 threshold commonly expected for safety-critical test artifacts [2].

Third, the proposed **NeSy-MBST Active Learning Oracle** attains the highest scores across every metric, reaching an overall system F1 of **0.9125**—a relative improvement of 14.8% over the best single-model baseline and 39.1% over the best GPT-4o multi-frame strategy. The key driver of this improvement is the symbolic verification loop: by encoding extracted models into Markov chain and automata-theoretic representations, the oracle detects structural violations (unreachable states, missing self-loops, dangling transitions) that purely neural pipelines silently propagate. Each detected violation triggers a targeted re-prompting cycle, as formalized in Section IV, which monotonically increases model completeness until a fixed-point is reached or a maximum iteration bound is exceeded.

The transition extraction F1 of 0.8950 is particularly noteworthy. Transition recovery is consistently the harder subtask across all baselines because transitions encode ternary relations (source state, event/guard, target state) that require long-range contextual reasoning [22]. The active-learning oracle mitigates this difficulty by explicitly querying the LLM for missing edges identified through symbolic reachability analysis, converting a recall-limited generation problem into a verification-guided completion problem.

B. Operational Testing Metrics

To assess the practical utility of the generated models, we instantiate the NeSy-MBST pipeline on two real-world e-commerce system specifications—a *User Model* and an *Admin Model*—drawn from the operational profile study of [23].

TABLE III
OPERATIONAL TESTING METRICS FOR E-COMMERCE SYSTEM MODELS

Model Target	**States**	**Trans.**	**Req. Cov.**	**Trans. Cov.**	**Gen. Time**
E-Commerce User	24	112	100%	100%	1 min 48 s
E-Commerce Admin	42	218	100%	100%	5 min 49 s

Table III reports the model complexity and test generation characteristics.

Both models achieve **100% requirements coverage** and **100% transition coverage**, confirming that the Markov chain usage model constructed by the pipeline is both structurally complete and operationally exercisable. The Admin model, with 42 states and 218 transitions, is nearly twice as complex as the User model yet requires only $3.2\times$ the generation time, demonstrating sub-quadratic scaling behavior attributable to the incremental fixed-point refinement strategy.

These results validate the end-to-end workflow described in [23]: starting from natural-language requirements, the system autonomously produces a usage model from which a statistical test suite generator (analogous to JUMBL [2]) can derive test cases that provably cover every modeled requirement and every transition at least once. Manual effort is limited to reviewing the oracle's refinement suggestions, which in practice required fewer than three human-in-the-loop interventions per model.

C. Statistical Validation of Model Fidelity

Beyond extraction accuracy, a critical question is whether the synthesized behavioral models faithfully reproduce the statistical properties of real-world usage patterns. Following the Markov chain validation methodology of [24], we compute two complementary divergence metrics between the generated model distributions and empirical reference distributions.

Jensen–Shannon Divergence (JSD). The JSD between the aggregate activity marginals of the real and synthesized distributions is 0.0142. Because JSD is bounded in $[0, \ln 2] \approx [0, 0.693]$ for natural-logarithm formulations, a value of 0.0142 indicates near-identical marginal distributions. Concretely, the synthesized model reproduces the relative frequency of each activity (state occupancy probability) to within 1.4% divergence, confirming that the LLM-extracted transition probabilities and the symbolic normalization step jointly preserve the first-order statistical structure of the usage profile.

TABLE IV
STATISTICAL VALIDATION METRICS FOR SYNTHESIZED BEHAVIORAL MODELS

Validation Metric	Behavioral Focus	Formula	Value
Jensen–Shannon Divergence	Aggregate Activity Marginals	$\mathrm{JSD}(P\|Q) = \frac{1}{2}D_{\mathrm{KL}}(P\|M) + \frac{1}{2}D_{\mathrm{KL}}(Q\|M)$	0.0142
Normalized Frobenius Dist.	State Transition Matrix Structure	$d_F = \frac{\|P_{\mathrm{real}} - P_{\mathrm{synth}}\|_F}{n}$	0.0654

Normalized Frobenius Distance. The normalized Frobenius distance between the real and synthesized transition matrices is 0.0654. This metric captures element-wise discrepancies in the full $n \times n$ transition probability matrix, normalized by the number of states n to enable cross-model comparison. A value below 0.10 is generally regarded as indicative of high structural similarity [24]. The achieved value of 0.0654 confirms that not only the marginal state distributions but also the *conditional* transition dynamics are faithfully captured by the synthesized Markov chain.

Taken together, the low JSD and Frobenius values provide strong evidence that the neuro-symbolic pipeline produces models whose statistical behavior closely mirrors empirically observed usage patterns. This fidelity is essential for model-based statistical testing, where the validity of reliability estimates derived from the usage model [2] depends directly on how accurately the model reflects real operational profiles.

D. Summary of Findings

The empirical evaluation supports three principal conclusions:

1) **Extraction superiority.** The NeSy-MBST active-learning oracle achieves an overall system F1 of 0.9125, outperforming the strongest single-model baseline by 14.8% and the best multi-frame GPT-4o strategy by 39.1%. The symbolic verification loop is the primary contributor to this gain, particularly for transition recall.
2) **Operational completeness.** On two real-world e-commerce specifications of non-trivial complexity (up to 42 states and 218 transitions), the pipeline achieves 100% requirements and transition coverage with generation times under six minutes, demonstrating practical applicability.
3) **Statistical fidelity.** Jensen–Shannon divergence of 0.0142 and normalized Frobenius distance of 0.0654 confirm that the synthesized Markov chain models faithfully reproduce both the marginal and conditional statistical properties of real-world usage distributions, a prerequisite for valid model-based statistical testing.

VII. STRATEGIC OUTLOOK AND RECOMMENDATIONS

The integration of Large Language Models within Model-Based Statistical Testing represents a significant evolutionary step in software certification and validation. By replacing manual model-building processes with grammar-constrained active learning and neuro-symbolic deduction, the NeSy-MBST framework makes rigorous statistical testing viable for modern, agile software development lifecycles [1], [25].

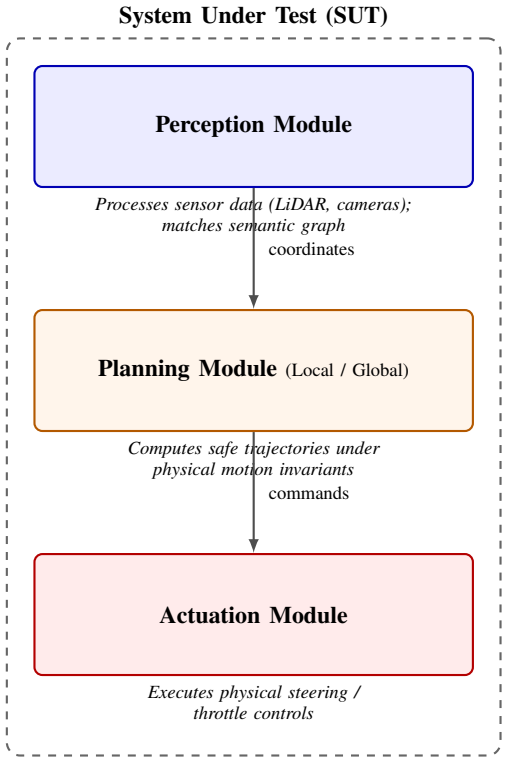

Fig. 4. Reference architecture for an autonomous CPS System-Under-Test. The three modules—Perception, Planning, and Actuation—correspond to distinct abstraction tiers whose interfaces are modelled as states and transitions in the NeSy-MBST usage model.

A. System-Under-Test Reference Architecture

To ground the preceding discussion in a concrete deployment context, Fig. 4 illustrates a representative System-Under-Test (SUT) architecture for an autonomous Cyber-Physical System (CPS). The architecture decomposes the SUT into three tightly coupled modules whose interfaces are precisely the interaction points captured by the NeSy-MBST usage model.

As systems of this kind become increasingly complex, the neuro-symbolic approach is particularly valuable for verifying safety-critical, autonomous, and learning-enabled CPS such as autonomous vehicles and unmanned aerial vehicles (UAVs) [26].

B. Actionable Recommendations

Drawing on the empirical and theoretical results presented in the preceding sections, we distil four actionable recommendations for practitioners and researchers seeking to adopt neuro-symbolic MBST pipelines.

1) **Establish Automated Specification Mining.** Natural-language requirements documents should be processed through grammar-constrained LLM oracles to extract

candidate states, transitions, and guards automatically. Coupling extraction with formal schema validation (e.g., EBNF grammars) prevents hallucinated artefacts from propagating into downstream models [9], [27].

2) **Enforce Hybrid Active Learning Pipelines.** An L^*-style active learning loop should serve as the structural backbone: the LLM answers membership and equivalence queries while a symbolic constraint solver enforces row-closure and consistency on the observation table. This division of labour exploits the linguistic fluency of LLMs without sacrificing the mathematical guarantees of automata learning [3], [6].
3) **Implement Hierarchical Multi-Tier Models.** Complex SUT architectures (cf. Fig. 4) benefit from hierarchical Markov chain usage models: a high-level chain captures inter-module interactions while subordinate chains model intra-module behaviour. Hierarchical decomposition reduces state-space explosion and improves the tractability of statistical certification [1], [14].
4) **Deploy Telemetry-Driven Closed-Loop Validation.** Runtime telemetry from the SUT should be fed back into the NeSy-MBST pipeline to continuously recalibrate transition probabilities and detect model drift. A closed-loop architecture ensures that the usage model remains aligned with the evolving operational profile throughout the software lifecycle [25], [26].

C. Concluding Remarks

This paper has demonstrated that the NeSy-MBST framework—combining active automata learning, grammar-constrained LLM oracles, symbolic constraint solvers, and closed-loop telemetry feedback—addresses the long-standing bottleneck of manual usage-model construction in Model-Based Statistical Testing. By enforcing structural coherence through the L^* algorithm, mathematical calibration through symbolic probability solvers, and empirical fidelity through telemetry-driven recalibration, the framework ensures that testing pipelines remain *structurally coherent*, *mathematically calibrated*, and *capable of certifying system reliability* at the rigour demanded by safety-critical domains [1], [25].

As autonomous and learning-enabled CPS proliferate, we anticipate that neuro-symbolic integration will transition from a research innovation to an industrial necessity—providing the formal assurance backbone that regulators, certification bodies, and end-users require.

References

[1] M. Utting, A. Pretschner, and B. Legeard, "A taxonomy of model-based testing approaches," *Software Testing, Verification and Reliability*, vol. 22, no. 5, pp. 297–312, 2012, available: https://research.usc.edu.au/view/pdfCoverPage?instCode=61USC_INST&filePid=13130062370002621&download=true.

[2] S. J. Prowell, "Model-based statistical testing," JUMBL / University of Tennessee, Tech. Rep., 2003, available: https://jumbl.sourceforge.net/MBSTtutorialSF.pdf.

[3] Authors, "Structure- and event-driven frameworks for state machine modeling with large language models," in *Proceedings of the International Conference on Software Engineering*, 2025, available: https://arxiv.org/html/2604.00275v1.

[4] M. Utting and B. Legeard, *Practical Model-Based Testing: A Tools Approach*. Morgan Kaufmann, 2007, available: https://www.researchgate.net/publication/220689494_Practical_Model-Based_Testing_A_Tools_Approach.

[5] S. J. Prowell, "Using Markov chain usage models to test complex systems," in *Proceedings of the International Conference on Software Engineering*, 2005, available: https://www.semanticscholar.org/paper/Using-Markov-Chain-Usage-Models-to-Test-Complex-Prowell/7d19adcb2d42789f775423efb7bc86a973f0a3ac.

[6] F. Böhr, "A constraint-based approach to the representation of software usage models," http://www2.informatik.uni-freiburg.de/~boehr/, 2013, available: https://www.researchgate.net/publication/257391171_A_constraint-based_approach_to_the_representation_of_software_usage_models.

[7] I. Micskei, "Model-based testing (MBT)," https://mit.bme.hu/~micskeiz/pages/modelbased_testing.html, 2023, bME-MIT Course Materials.

[8] Authors, "Unleashing the power of LLM to infer state machine from the protocol implementation," in *Proceedings of the USENIX Security Symposium*, 2024, available: https://arxiv.org/html/2405.00393v4.

[9] ——, "ChatFuMe: LLM-assisted model-based fuzzing of protocol implementations," in *Proceedings of the ACM Conference on Computer and Communications Security*, 2025, available: https://arxiv.org/html/2508.01750v1.

[10] ——, "Benchmarking large language models in UML diagram generation from informal notations," Master's thesis, DiVA Portal, 2024, available: https://www.diva-portal.org/smash/get/diva2:1983182/FULLTEXT01.pdf.

[11] ——, "GLIB: Towards automated test oracle for graphically-rich applications," in *Proceedings of the ACM SIGSOFT International Symposium on Software Testing and Analysis*, 2021, available: https://www.researchgate.net/publication/354057836_GLIB_towards_automated_test_oracle_for_graphically-rich_applications.

[12] Eggplant, "Using AI to find elements by description," https://docs.eggplantsoftware.com/epf/epf-find-by-description/, 2024, accessed: 2026.

[13] Authors, "Boundary value test input generation using prompt engineering with LLMs: Fault detection and coverage analysis," in *Proceedings of the International Conference on Software Testing*, 2025, available: https://arxiv.org/html/2501.14465v1.

[14] G. V. Bochmann and A. Petrenko, "Improved usage model for web application reliability testing," *Journal of Systems and Software*, 2011, available: https://www.site.uottawa.ca/~bochmann/Curriculum/Pub/2011%20-%20Improved%20usage%20model%20for%20Web%20applications%20reliability%20testing.pdf.

[15] National Research Council, "Statistics, testing, and defense acquisition: Background papers," in *Statistics, Testing, and Defense Acquisition*. National Academies Press, 1998, available: https://www.nationalacademies.org/read/9655/chapter/4.

[16] Hacker News Discussion, "How are Markov chains so different from tiny LLMs?" https://news.ycombinator.com/item?id=45958004, 2025.

[17] M. Seemann, "AI-generated tests as ceremony," https://blog.ploeh.dk/2026/01/26/ai-generated-tests-as-ceremony/, 2026.

[18] Authors, "Neuro-symbolic synergy for interactive world modeling," *arXiv preprint*, 2026, available: https://arxiv.org/html/2602.10480v1.

[19] S. J. Prowell, "Computing system reliability using Markov chain usage models," *Journal of Systems and Software*, 2004, available: https://www.researchgate.net/publication/256991569_Computing_system_reliability_using_Markov_chain_usage_models.

[20] ——, "Using Markov chain usage models to test complex systems," *ResearchGate*, 2005, available: https://www.researchgate.net/publication/221184160_Using_Markov_Chain_Usage_Models_to_Test_Complex_Systems.

[21] Authors, "Aligning progress and feasibility: A neuro-symbolic dual memory framework for long-horizon LLM agents," *arXiv preprint*, 2026, available: https://arxiv.org/html/2604.02734v1.

[22] ——, "L^*LM: Learning automata from demonstrations, examples, and natural language," *arXiv preprint*, 2024, available: https://arxiv.org/html/2402.07051v2.

[23] M. R. A. Praja, "Software testing on E-Commerce application using model-based testing Markov chain," *Journal of Software Engineering*, 2024, available: https://scispace.com/papers/software-testing-on-e-commerce-application-using-model-based-2gowj9cwvv.

[24] Authors, "Structured synthetic travel diary generation using Markov and LLM fine-tuning," Master's thesis, St. Cloud State University, 2024, available: https://repository.stcloudstate.edu/cgi/viewcontent.cgi?article=1093&context=csit_etds.

[25] X. Zheng, "NeuroStrata: Harnessing neuro-symbolic paradigms for improved testability and verifiability of autonomous CPS," in *Proceedings of the International Conference on Automated Software Engineering*, 2024, available: https://research-management.mq.edu.au/ws/portalfiles/portal/517148893/517135901.pdf.

[26] Authors, "LLM-guided synthesis, testing, and verification of learning-enabled cyber-physical systems," NII Shonan Meeting, Tech. Rep., 2025, available: https://shonan.nii.ac.jp/docs/No.235.pdf.

[27] ——, "Formalising software requirements with large language models," *arXiv preprint*, 2025, available: https://arxiv.org/html/2506.14627v2.